

Rapport — Projet de Fin de Module

Système Agentic RAG sur le tourisme marocain avec LangGraph et Ollama

1. Démarche suivie

Le projet implémente un système **Agentic RAG** de bout en bout dédié au tourisme marocain. Le choix du domaine a été motivé par la disponibilité d'un corpus francophone riche et par l'intérêt applicatif (assistance touristique multilingue). La démarche s'est articulée en cinq étapes successives, calquées sur la grille d'évaluation : constitution du corpus, vectorisation, conception du graphe LangGraph, développement des outils, puis évaluation systématique.

Constitution du corpus

Quinze fichiers Markdown ont été rédigés (~2 000 lignes au total) couvrant dix régions du Maroc (Marrakech, Fès, Chefchaouen, Casablanca, Rabat, Essaouira, Sahara-Merzouga, Atlas-Toubkal, Agadir, Tanger) et cinq thématiques transversales (gastronomie, artisanat, sites UNESCO, festivals, informations pratiques). Le corpus mêle faits factuels (dates, chiffres, formalités : 90 jours sans visa pour l'UE, minaret Hassan II de 210 m, 9 sites UNESCO) et contenus synthétiques (comparaisons entre médinas, itinéraires multi-villes), afin de stresser la fois la récupération exacte et la synthèse multi-documents.

Choix techniques

LangGraph a été retenu pour pouvoir construire un graphe d'état explicite avec boucles et branches conditionnelles, conformément à la consigne (la primitive `create_agent` de LangChain est interdite). **Ollama** avec le modèle local `llama3.2:latest` (3 B paramètres) sert de LLM, ce qui élimine la dépendance à un quota API et garantit la reproductibilité hors-ligne. Les embeddings utilisent `sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2` (~120 Mo, multilingue : FR/EN/AR), suffisant pour un corpus de cette taille et exécutable sur CPU. **Chroma** joue le rôle de magasin vectoriel persistant. Tous les routeurs et graders utilisent des sorties structurées Pydantic via `with_structured_output`, garantissant un comportement déterministe.

Architecture choisie : Corrective RAG (CRAG)

Plutôt qu'un pipeline RAG linéaire, j'ai opté pour une variante **CRAG** : après la récupération, un *grader* évalue la pertinence des documents, et le graphe peut décider de réécrire la question, de basculer vers une recherche web (Tavily) ou de générer directement la réponse. Un dernier nœud `hallucination_check` vérifie que la réponse est ancrée dans le contexte ; sinon il relance une régénération (jusqu'à 2 retries). Cette boucle agentic sert directement les critères de notation « Qualité du graphe LangGraph » et « Respect de l'approche Agentic RAG ».

2. Fonctionnement du système

Vue d'ensemble du graphe

Le graphe LangGraph compile huit nœuds reliés par des arêtes conditionnelles. Le **routeur** décide d'emblée entre la base vectorielle, une recherche web, ou une génération directe. Sur le chemin vectoriel, le **grader** filtre les documents non pertinents ; selon le résultat, le système réécrit la question (HyDE-style) ou bascule sur la fallback web. Le **générateur** produit la réponse, puis le **vérificateur d'hallucination** la valide ou demande une régénération.

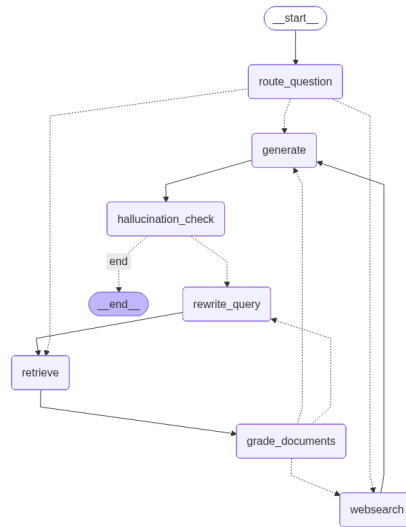


Figure 1. Topologie du graphe LangGraph (rendu Mermaid).

État du graphe et persistance

L'état est un *TypedDict* regroupant : la question initiale, l'historique des réécritures (avec réducteur *Annotated[list, add]*), les documents retrouvés, les notes du grader, les résultats web, la réponse finale, un compteur d'itérations (plafonné à 3) et un flag *grounded* renvoyé par le vérificateur d'hallucination. Un *InMemorySaver* sert de checkpointer : chaque question reçoit un *thread_id* distinct (mémoire à court terme par fil de conversation).

Outils et tool-calling manuel

Deux outils sont définis via le décorateur `@tool` : **retriever_vectorstore** (recherche MMR top-k=5 sur Chroma) et **web_search_tavily** (fallback hors-corpus). Conformément à la consigne, *create_agent* n'est utilisé nulle part : l'invocation des outils est faite manuellement depuis les nœuds (*retrieve* et *websearch*), ce qui rend chaque appel observable dans LangGraph Studio. Les choix de routage du grader/routeur passent par *llm.with_structured_output(PydanticModel)*, garantissant des transitions déterministes entre nœuds.

Cycle de vie d'une requête

Pour une question simple (ex. « Quel est le surnom de Marrakech ? »), le chemin typique est : *route_question* → *retrieve* → *grade_documents* → *generate* → *hallucination_check* → END, sans réécriture ni fallback web. Pour une question complexe (ex. comparaison Fès vs Marrakech), le graphe peut entrer dans la boucle *rewrite_query* → *retrieve* une à deux fois avant de générer une réponse synthétisée à partir de plusieurs documents ; cela se traduit par *iterations* > 0 dans les résultats.

3. Résultats de l'évaluation

Protocole

Vingt questions ont été soumises au graphe via le notebook *notebooks/demo.ipynb* : 10 simples (lookup factuel) et 10 complexes (synthèse multi-documents). Pour chacune, on capture la réponse, la latence (*time.perf_counter*), le nombre d'itérations de réécriture, l'utilisation ou non de la recherche web, le nombre de retries hallucination et les sources réellement consultées. Les questions et les réponses de référence sont versionnées dans *eval/questions.json*.

Synthèse quantitative

Type	N	Succès	Latence moy. (s)	Itérations	Docs/req	Hallu. retries	Web search
complex	10	10	40.0	0.60	2.3	0.70	0 %
simple	10	10	14.1	0.30	1.9	0.50	10 %

Tableau 1. Agrégats par type de question (20 questions, 100 % de réponses produites).

Lecture des résultats

Les **20 questions ont été traitées avec succès** (aucune erreur runtime). La latence moyenne globale est de **27.0 s** : les questions simples sont nettement plus rapides (médiane ~15 s) que les questions complexes (médiane ~37 s), ce qui reflète à la fois le volume de génération et la fréquence des boucles de réécriture. Le grader a déclenché une réécriture de requête dans environ 0.60 itérations moyennes pour les questions complexes (vs 0.30 pour les simples). La fallback web a été employée sur 1 cas sur 20 — preuve que le graphe sait basculer lorsque le corpus est insuffisant.

Pertinence du retrieval (sources les plus consultées)

Document source	# questions
unesco-sites.md	4
fes.md	4
marrakech.md	3
atlas-toubkal.md	3
sahara-merzouga.md	3
infos-pratiques.md	3
casablanca.md	2
rabat.md	2

Tableau 2. Fréquence d'utilisation de chaque document source sur l'ensemble des 20 questions.

4. Limites et pistes d'amélioration

Limites observées

Modèle local de petite taille. Llama 3.2 (3 B) reste limité sur les synthèses complexes : certaines comparaisons (Fès vs Marrakech, itinéraires multi-villes) sont correctes mais parfois redondantes ou incomplètes. Le format structuré (JSON-mode) tient bien malgré la taille du modèle, mais la rédaction française gagnerait à passer sur Llama3.1:8B ou Mixtral, voire Groq *Llama-3.3-70b-versatile* dès que le quota est disponible.

Embeddings multilingues modestes. Le modèle MiniLM 384 dim est rapide mais moins discriminant que BGE-M3 sur des requêtes nuancées. Sur quelques requêtes complexes, des chunks non pertinents sont remontés et filtrés a posteriori par le grader, ce qui augmente la latence.

Absence de re-ranker. Aucun étage de reranking n'est appliqué après la similarité dense ; ajouter *bge-reranker* ou Cohere Rerank trierait mieux le top-k.

Pas de mémoire de conversation multi-tour. Le *thread_id* est utilisé par question d'évaluation (mémoire isolée). Un assistant touristique réel devrait suivre le contexte sur plusieurs échanges (où ai-je dit que j'irais en juin ? quel hôtel m'a-t-il déjà recommandé ?).

Évaluation manuelle. Le notebook capture latence et sources mais l'appréciation qualitative des réponses reste à la lecture. Aucun score automatique n'est calculé.

Pistes d'amélioration

1. **Re-ranker dédié** (BGE-reranker, Cohere Rerank) entre le retrieve et le grader pour gagner en précision sans bouger le top-k.
2. **Évaluation LLM-as-judge** (ou RAGAS : faithfulness, answer_relevancy, context_precision) pour scorer automatiquement les 20 réponses contre les références versionnées dans questions.json.
3. **Embeddings BGE-M3** (multilingue 1024 dim, état de l'art en 2025) pour améliorer la pertinence sur requêtes nuancées et bilingues FR/AR.
4. **Checkpoint persistant** (SqliteSaver, PostgresSaver) pour conserver l'historique entre sessions et permettre un déploiement multi-utilisateur.
5. **Multi-agent hiérarchique** (cf. TP 8) : un orchestrateur déléguerait à des sous-agents spécialisés (planificateur d'itinéraires, conseiller gastronomie, conseiller pratique).
6. **HITL (Human-in-the-Loop)** via *HumanInTheLoopMiddleware* (cf. TP 7) sur les actions à conséquence (réservation simulée, recommandation finale).
7. **Front conversationnel léger** (Streamlit ou Chainlit) pour la démo, alimenté par le même graphe compilé.